

Theory – CLRS

1 – Role of Algorithms

An algorithm is any well-defined computational procedure that takes some value, or set of values, as input and produces some value, or set of values, as output in a finite amount of time.

An algorithm for a computational problem is correct if, for every problem instance provided as input, it halts (finishes its computing in finite time) and outputs the correct solution to the problem instance. A correct algorithm solves the given computational problem.

NP – complete problems: don't have any efficient algorithm that solves them. If an algo solves one, then it can be proved that there are algos for each and every other NP-complete problem. Instead, we use 'approximation' algos for this case.

2 – Getting started

Insertion sort

Our first algorithm, insertion sort, solves the sorting problem introduced in Chapter 1:

Input: A sequence of n numbers ($a_1; a_2; \dots; a_n$).

Output: A permutation (reordering) ($a_{01}; a_{02}; \dots; a_{0n}$) of the input sequence such that $a_{01} \leq a_{02} \leq \dots \leq a_{0n}$.

The numbers to be sorted are also known as the keys. Although the problem is conceptually about sorting a sequence, the input comes in the form of an array with n elements. When we want to sort numbers, it's often because they are the keys associated with other data, which we call satellite data. Together, a key and satellite data form a record.

```
INSERTION-SORT( $A, n$ )
1  for  $i = 2$  to  $n$ 
2       $key = A[i]$ 
3      // Insert  $A[i]$  into the sorted subarray  $A[1 : i - 1]$ .
4       $j = i - 1$ 
5      while  $j > 0$  and  $A[j] > key$ 
6           $A[j + 1] = A[j]$ 
7           $j = j - 1$ 
8       $A[j + 1] = key$ 
```

Here as we iterate through the array, the elements in the subarray before a specific element ($A[i]$) i.e. $A[1:i-1]$, are already sorted (initial case being $A[1]$ which is the starting element), then the while loop looks at $A[i]$ and decides where to put it in the sorted subarray by shifting the elements greater than it in the subarray one index ahead, till it reaches $A[j] \leq key$.

'Loop invariants' help us understand why an algorithm is correct. When you're using a loop invariant, you need to show three things:

Initialization: It is true prior to the first iteration of the loop.

Maintenance: If it is true before an iteration of the loop, it remains true before the next iteration.

Termination: The loop terminates, and when it terminates, the invariant usually along with the reason that the loop terminated gives us a useful property that helps show that the algorithm is correct.

Using the loop invariant to show that the insertion algo is correct; understand it properly!

Initialization:

We start by showing that the loop invariant holds before the first loop iteration, when $i = 2$. The subarray $A[1:i-1]$ consists of just the single element $A[1]$, which is in fact the original element in $A[1]$. Moreover, this subarray is sorted (after all, how could a subarray with just one value not be sorted?)

Maintenance:

Next, we tackle the second property: showing that each iteration maintains the loop invariant. Informally, the body of the for loop works by moving the values in $A[i-1]$, $A[i-2]$, $A[i-3]$, and so on by one position to the right until it finds the proper position for $A[i]$ (which is done by the while loop), at which point it inserts the value of $A[i]$ (line 8). The subarray $A[1:i]$ then consists of the elements originally in $A[1:i]$, but in sorted order. Incrementing i (increasing its value by 1) for the next iteration of the for loop then preserves the loop invariant.

Termination:

Finally, we examine loop termination. The loop variable i starts at 2 and increases by 1 in each iteration. Once i 's value exceeds n in line 1, the loop terminates. That is, the loop terminates once i equals $n+1$. Substituting $n+1$ for i in the wording of the loop invariant yields that the subarray $A[1:n]$ consists of the elements originally in $A[1:n]$, but in sorted order. Hence, the algorithm is correct.

Analysing Algorithms

Analysing an algorithm has come to mean predicting the resources that the algorithm requires. You might consider resources such as memory, communication bandwidth, or energy consumption. Most often, however, you'll want to measure computational time.

In the RAM model of computation, instructions execute one after another, with no concurrent operations i.e., each instruction or data access takes a constant amount of time⁴even indexing into an array. Here, we assume that each line of code takes a constant amount of time to get executed.

Obviously, the running time of an algo depends on its input size. The running time of an algorithm on a particular input is the number of instructions and data accesses executed.

The running time of the algorithm is the sum of running times for each statement executed. A statement that takes c_k steps to execute and executes m times contributes $c_k m$ to the total running time. We usually denote the running time of an algorithm on an input of size n by $T(n)$.

Note that the measurement of 'input size' depends on the problem being studied - the most natural measure is the number of items in the input - for example, the number n of items being sorted. For many other problems, such as multiplying two integers, the best measure of input size is the total number of bits needed to represent the input in ordinary binary notation.

Analysing Insertion Sort

For each $i = 2, 3, \dots, n$, let t_i denote the number of times the while loop test in line 5 is executed for that value of i . IMPORTANT - When a for or while loop exits in the usual way, because the test in the loop header comes up FALSE, the test is executed one time more than the loop body (since in that case the loop body doesn't get executed). Because comments are not executable statements, assume that they take no time.

INSERTION-SORT(A, n)	cost	times
1 for $i = 2$ to n	c_1	n
2 $key = A[i]$	c_2	$n - 1$
3 <i>// Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$.</i>	0	$n - 1$
4 $j = i - 1$	c_4	$n - 1$
5 while $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^n t_i$
6 $A[j + 1] = A[j]$	c_6	$\sum_{i=2}^n (t_i - 1)$
7 $j = j - 1$	c_7	$\sum_{i=2}^n (t_i - 1)$
8 $A[j + 1] = key$	c_8	$n - 1$

$$T(n) = c_1n + c_2(n - 1) + c_4(n - 1) + c_5 \sum_{i=2}^n t_i + c_6 \sum_{i=2}^n (t_i - 1) + c_7 \sum_{i=2}^n (t_i - 1) + c_8(n - 1).$$

In the best-case scenario, the input array A is already sorted, so that the while loop's second condition is false on the first try itself. Thus, the best-case running time is:

$$\begin{aligned} T(n) &= c_1n + c_2(n - 1) + c_4(n - 1) + c_5(n - 1) + c_8(n - 1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8). \end{aligned}$$

We can see that this is a linear function of n .

In the worst-case scenario, the input array A is already sorted in descending order; so here the second while loop condition is true for each preceding i 's; thus $t_i = i$ (The procedure finds that $A[j] > key$ every time in line 5, and the while loop exits only when j reaches 0). Thus, running time is:

$$\begin{aligned} T(n) &= c_1n + c_2(n - 1) + c_4(n - 1) + c_5 \left(\frac{n(n + 1)}{2} - 1 \right) \\ &\quad + c_6 \left(\frac{n(n - 1)}{2} \right) + c_7 \left(\frac{n(n - 1)}{2} \right) + c_8(n - 1) \\ &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n \\ &\quad - (c_2 + c_4 + c_5 + c_8). \end{aligned}$$

Here, we have a quadratic function of n .

We consider worst-case scenarios since it gives an upper bound on the running time of an algo.

Designing Algorithms

Insertion sort uses the incremental method, which is one method of algorithm design.

Divide and Conquer Method & Merge Sort

Many useful algorithms are recursive in structure: to solve a given problem, they recurse (call themselves) one or more times to handle closely related subproblems. These algorithms typically follow the divide-and-conquer method: they break the problem into several subproblems that are similar to the original problem but smaller in size, solve the subproblems recursively, and then combine these solutions to create a solution to the original problem.

Merge sort is one such algo that follows this method - In each step, it sorts a subarray $A[p:r]$, starting with the entire array $A[1:n]$ and recursing down to smaller and smaller subarrays. Steps:

Divide the subarray $A[p:r]$ to be sorted into two adjacent subarrays, each of half the size. To do so, compute the midpoint q of $A[p:r]$ (taking the average of p and r), and divide $A[p:r]$ into subarrays $A[p:q]$ and $A[q+1:r]$.

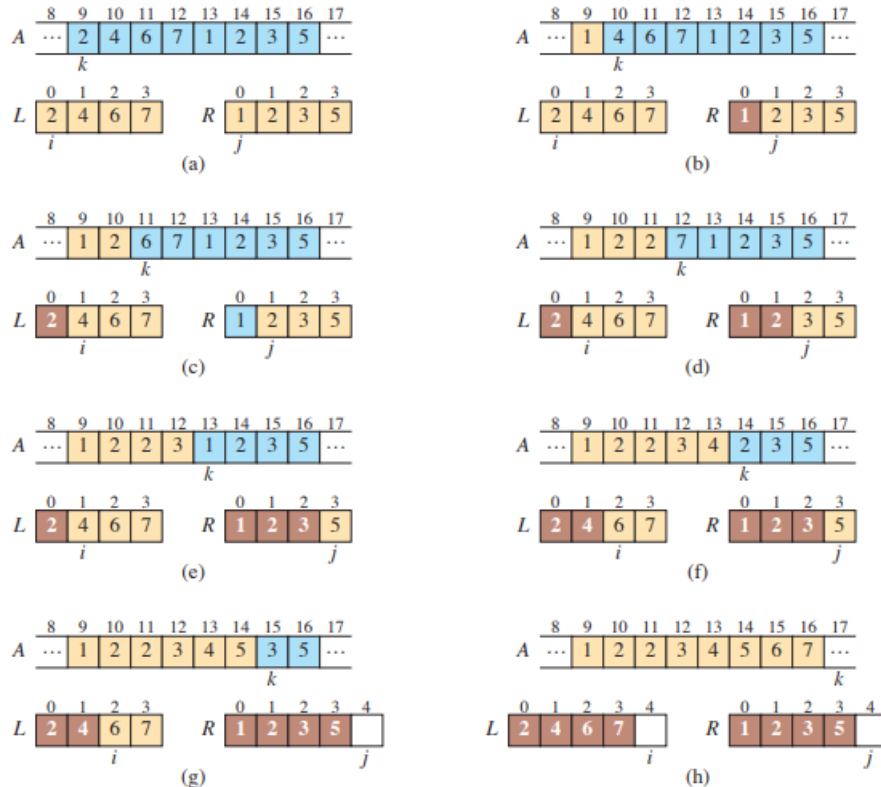
Conquer by sorting each of the two subarrays $A[p:q]$ and $A[q+1:r]$ recursively using merge sort i.e., again dividing them repeatedly.

Combine by merging the two sorted subarrays $A[p:q]$ and $A[q+1:r]$ back into $A[p:r]$, producing the sorted answer.

In the divide-and-conquer method, if the problem is small enough - the base case - you just solve it directly without recursing. In merge sort, it 'bottoms out' i.e. reaches the base case when the subarray $A[p:r]$ has just 1 element. Example:

```
MERGE( $A, p, q, r$ )
1   $n_L = q - p + 1$       // length of  $A[p:q]$ 
2   $n_R = r - q$           // length of  $A[q + 1:r]$ 
3  let  $L[0:n_L - 1]$  and  $R[0:n_R - 1]$  be new arrays
4  for  $i = 0$  to  $n_L - 1$  // copy  $A[p:q]$  into  $L[0:n_L - 1]$ 
5       $L[i] = A[p + i]$ 
6  for  $j = 0$  to  $n_R - 1$  // copy  $A[q + 1:r]$  into  $R[0:n_R - 1]$ 
7       $R[j] = A[q + j + 1]$ 
8   $i = 0$                 //  $i$  indexes the smallest remaining element in  $L$ 
9   $j = 0$                 //  $j$  indexes the smallest remaining element in  $R$ 
10  $k = p$                 //  $k$  indexes the location in  $A$  to fill
11 // As long as each of the arrays  $L$  and  $R$  contains an unmerged element,
    // copy the smallest unmerged element back into  $A[p:r]$ .
12 while  $i < n_L$  and  $j < n_R$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
18      $k = k + 1$ 
19 // Having gone through one of  $L$  and  $R$  entirely, copy the
    // remainder of the other to the end of  $A[p:r]$ .
20 while  $i < n_L$ 
21      $A[k] = L[i]$ 
22      $i = i + 1$ 
23      $k = k + 1$ 
24 while  $j < n_R$ 
25      $A[k] = R[j]$ 
26      $j = j + 1$ 
27      $k = k + 1$ 
```

As seen in the next diagram, steps 12-18 first compare the respective first elements of the L and R arrays and add the lesser one back to the main array to be sorted; then it advances one index in the L/R array (the one from which the element was taken) and then compares the element at that index with the first element of the L/R array that had the greater element in the first step; and so on until its over. Then when one of L/R has been exhausted, lines 20-27 add the remaining elements from the L/R (whichever remains) array to the main array.



To see that the MERGE procedure runs in $f(n)$ time, where $n = r - p + 1$ observe that each of lines 1-3 and 8-10 takes constant time, and the for loops of lines 4-7 take $f(n_L + n_R) = f(n)$ time. To account for the three while loops of lines 12-18, 20-23, and 24-27, observe that each iteration of these loops copies exactly one value from L or R back into A and that every value is copied back into A exactly once. Therefore, these three loops together make a total of n iterations. Since each iteration of each of the three loops takes constant time, the total time spent in these three loops is $f(n)$!

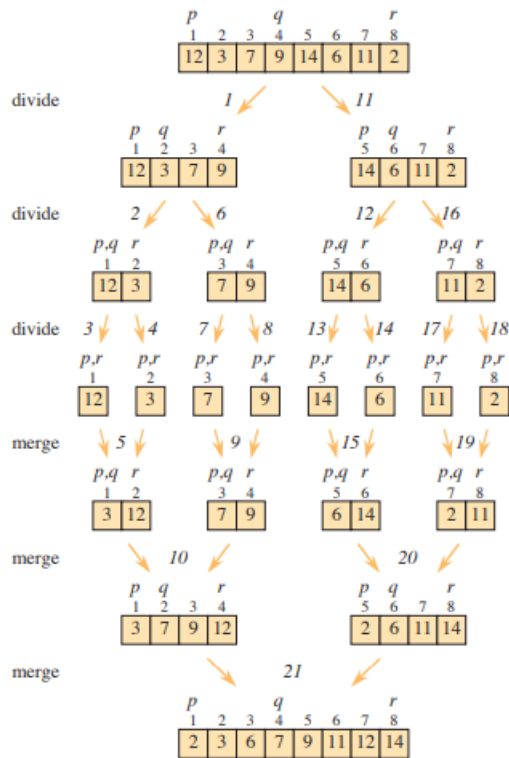
Now, the function we described above can be used as a subroutine (remember, subroutine is just a function but it doesn't return anything; it just performs a task) recursively as shown to sort the entire array – it keeps on breaking down the array into halves until it's just one element each in the L and R subarrays, then does the comparison as in the previous program and starts building back up again:

```

MERGE-SORT( $A, p, r$ )
1  if  $p \geq r$                                 // zero or one element?
2      return
3   $q = \lfloor (p + r) / 2 \rfloor$                     // midpoint of  $A[p : r]$ 
4  MERGE-SORT( $A, p, q$ )                        // recursively sort  $A[p : q]$ 
5  MERGE-SORT( $A, q + 1, r$ )                    // recursively sort  $A[q + 1 : r]$ 
6  // Merge  $A[p : q]$  and  $A[q + 1 : r]$  into  $A[p : r]$ .
7  MERGE( $A, p, q, r$ )

```

For example, as shown here with an array of size $n=8$:



Analysing Divide and Conquer Algorithms

When an algorithm contains a recursive call, you can often describe its running time by a recurrence equation or recurrence, which describes the overall running time on a problem of size n in terms of the running time of the same algorithm on smaller inputs.

Let $T(n)$ be the worst case running time on a problem of size ' n '; for the three parts of divide and conquer algos, we can identify three ways time is used:

Dividing: takes $D(n)$ time for an array of n elements

Conquering: Suppose the division yields ' a ' subproblems, each of size n/b (in merge sort, $a=b=2$) so that solving each subproblem takes $T(n/b)$ time so that total is $a \cdot T(n/b)$.

Combining: takes $C(n)$ time to combine all the solved subproblems to the original problem.

Thus, we can create a recurrence like this:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n < n_0, \\ D(n) + aT(n/b) + C(n) & \text{otherwise.} \end{cases}$$

Where n_0 is some constant below which the solving always takes a constant amount of time.

For merge sort specifically, putting $a=b=2$, we get $T(n) = 2T(n/2) + f(n)$, where $\Theta(n)$ is $D(n)+C(n)$; $D(n) = \Theta(1)$ since the divide step just computes the midpoint of the subarray and $C(n) = \Theta(n)$ since it merges everything back.

3 – Characterizing Running Times

When we look at input sizes large enough to make relevant only the order of growth of the running time, we are studying the asymptotic efficiency of algorithms. That is, we are concerned with how the running time of an algorithm increases with the size of the input in the limit, as the size of the input increases without bound. Usually, an algorithm that is asymptotically more efficient is the best choice for all but very small inputs.

Asymptotic Notations:

Θ -notation:

Here, we discard the lower-order terms and the coefficient of the leading term, and use a notation that focuses on the rate of growth of the running time. It characterizes a tight bound on the asymptotic behaviour of a function. It says that a function grows precisely at a certain rate, based on the highest-order term.

Theory – Google Dev Guide

Maps/Dictionaries

Quick Intro:

Maps (also known as Dictionaries) are data structures stores a collection of key-value pairs. Each key is unique and allows for quick access to values.

They hold data in a table structure that has two columns: a key, and a value. Hashmaps use a **hash function** to determine where to store the data; this **takes in a key and creates a hash code which determines where to store the value**, such that identical keys always point to the same value.

They use **put and get methods to store and retrieve data** respectively:

```
Students.put("1234", "Ruhaib"), and Students.get("1234")
```

Note: **If you add a value to a map/dictionary with a key that already exist, that new value will replace the original value.**

Values in a map/dictionary are unordered. There is no explicit beginning or end.

How they work:

C++ arrays store items of a single data type contiguously in memory (the elements' memory locations are next to each other in sequence); each element has its own memory location such that we can access them in random order. Because of this, we have to specify the array size upon declaration so that we can ask the OS to reserve that much memory for the array's elements. There's no guarantee that extra memory adjacent to these locations will be available later on to expand the array.

Hash tables -> when quick insertion, deletion and lookup are a priority -> achieved in $O(1)$

== array coupled with the hash function

This function takes in a key and outputs an integer called hash value that maps the key to a particular index in the table. Thus, for a given key:

1. First use the has function to determine where in the table to store a given key
2. Use same function to determine where in the table to search for a key

They output same hash values for the same keys i.e., hash functions generate specific indexes for keys

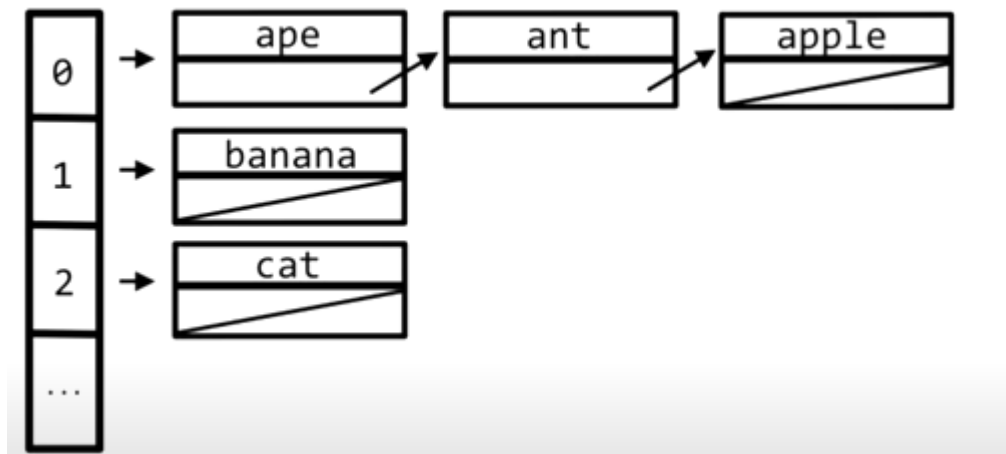
Collision: two keys hashing to the same index location

Two methods to resolve collisions:

1. Linear probing: if a key hashes to the same index as a previously stored key, it's assigned the next slot/index available i.e., with no value in the table.
leads to increased chances of collisions in the same area → "Clustering"; major drawback
And, insertion, deletion and look up times become $O(n)$ since the next available slot could

potentially be the last (nth) slot in the table.

2. Separate chaining: the table is an array of pointers to linked lists.



When a collision occurs, the key can be inserted at the head of the appropriate linked list instead. If the linked list has a length of k and there are total n values, worst case lookup becomes $O(n/k) \sim O(n)$; practically still a huge improvement though.

Thus we want a hash function to reduce chances of collisions. It must have the following properties:

- Makes use of all info provided by key
- Uniformly distributes output across table
- Maps similar keys to very different hash values
- Uses only very fast operations

[Leetcode Tutorial:](#)

Hash Table is a data structure which **organizes data using hash functions** in order to support quick insertion and search.

There are two different kinds of hash tables: hash set and hash map.

- The hash **set** is one of the implementations of a set data structure to **store no repeated values**.
- The hash **map** is one of the implementations of a map data structure to **store (key, value) pairs**.

Stacks & Queues

Both are linear data structures & have flexible sizes.

Stack: LIFO – like a stack of plates – last one u put on top is the first one you'll remove

Queue: FIFO – like a queue of people – first guy leaves the line first

Problems

Valid Parentheses

You are given a string *s* consisting of the following characters: '(', ')', '{', '}', '[' and ']'.

The input string *s* is valid if and only if:

1. Every open bracket is closed by the same type of close bracket.
2. Open brackets are closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Return true if *s* is a valid string, and false otherwise.

Example 1: Input: *s* = "()"; Output: true

Example 2: Input: *s* = "({})"; Output: true

Example 3: Input: *s* = "[()]"; Output: false

$O(n)$; since we're iterating through the string only once.

First, create a map from the closed parentheses to their corresponding open parentheses.

Then, create a stack that will store the opening parentheses, added in order one by one.

Next, the loop checks if the current character is a closing parentheses;. If not, it adds it to the stack in order. If yes, then we need to make sure that its corresponding open parentheses is the latest added member of the stack. If this is True, then this closing and open parentheses complete each other and hence can be removed from the stack. Finally if the stack is empty, then all the opening parentheses have closing parentheses in order, hence being valid.

```
class Solution:
    def isValid(self, s: str) -> bool:
        charMap = {"}": "{", "]": "[", "}": "("}
        stack = []
        for char in s:
            if char in charMap: #checks for closing parentheses
                if stack and stack[-1] == charMap[char]:
                    stack.pop()
                    #checks if last element added is corresponding open parentheses
                    #if yes, then pop it out; it's valid. Stack must be empty by end
                else:
                    return False #closing parentheses without opening one!
            else:
                stack.append(char) #adds opening chars one by one, in order
        return True if not stack else False
```

Graphs

Quick Intro:

Graphs = (V, E) = ordered pair of vertices and edges; a non-linear data structure

Vertices = $\{V_1, V_2, \dots\}$ and Edges = $\{E_1, E_2, \dots\}$ == unordered sets.

If fully connected graph, $n(E) = n(V) - 1$ (same as in trees?)

Edges can be directed or undirected. Eg: E_1 is edge b/w nodes a and b; then

$E_1 = (a, b) \rightarrow$ directed, from a to b; eg.: world wide web; one page leads to another.

$E_1 = \{a, b\} \rightarrow$ undirected; traversal in both ways is possible. Eg.: facebook friend network

Usually in graphs, all edges are either of one type.

All directed edges $\rightarrow$ 'directed graph' or 'digraph'

Edges can have weights associated with them.

Eg. Of undirected weighted graph: intercity road network; edges between closer cities will have lesser weights

Eg. Of directed weighted: intra(within)city road network.

Graph Search Algos

Consider a graph object defined as such:

```
# A class to represent a graph object
class Graph:
    # Constructor
    def __init__(self, edges, n):

        # A list of lists to represent an adjacency list
        self.adjList = [[] for _ in range(n)]

        # add edges to the undirected graph
        for (src, dest) in edges:
            self.adjList[src].append(dest)
            self.adjList[dest].append(src)
```

Depth-First Search

- Uses stack (LIFO) to store nodes/vertices
- Used to go deep into graph and find a path; backtracking, complete search, exhausting all possible paths
- Done recursively, until all adjacent nodes are seen
- Time complexity: $O(V+E)$; space complexity: $O(v)$ since stack will hold all vertices

Algo (iterative):

1. Create a stack to hold all the nodes; first push the starting node to it
 2. Create a hash set to contain all the nodes that are already visited/seen
 3. While loop for while the stack is not empty:
 - a. Pop last node out of the stack and check if it's in the seen set. If yes then pass to for loop
 - b. If no, then add it to the seen set, and 'process' it i.e., output it as a visited node
 - c. For loop, for each node adjacent to the current node:
 - i) Check if adjacent node is in seen set; if no then **push it to the stack**
 - ii) If it's already seen, then pass
- Now, the stack has all the adjacent nodes; the while loop handles this then.

```

# Perform iterative DFS on graph starting from vertex `v`
def iterativeDFS(graph, v, discovered):

    # create a stack used to do iterative DFS
    stack = deque()

    # push the source node into the stack
    stack.append(v)

    # loop till stack is empty
    while stack:

        # Pop a vertex from the stack
        v = stack.pop()

        # if the vertex is already discovered yet, ignore it
        if discovered[v]:
            continue

        # we will reach here if the popped vertex `v` is not discovered yet;
        # print `v` and process its undiscovered adjacent nodes into the stack
        discovered[v] = True
        print(v, end= ' ')

        # do for every edge (v, u)
        adjList = graph.adjList[v]
        for i in reversed(range(len(adjList))):
            u = adjList[i]
            if not discovered[u]:
                stack.append(u)

```

(Since no stack in python, we use <https://www.geeksforgeeks.org/deque-in-python/> instead)

Recursive DFS:

```

# Function to perform DFS traversal on the graph on a graph
def DFS(graph, v, discovered):

    discovered[v] = True          # mark the current node as discovered
    print(v, end= ' ')          # print the current node

    # do for every edge (v, u)
    for u in graph.adjList[v]:
        if not discovered[u]:    # if `u` is not yet discovered
            DFS(graph, u, discovered)

```

Interesting to note: You don't need to use a stack in DFS!

Applications of DFS:

- Finding connected components in a graph.
- Topological sorting in a DAG(Directed Acyclic Graph).
- Finding 2/3-(edge or vertex)-connected components.
- Finding the bridges of a graph.
- Finding strongly connected components.
- Solving puzzles with only one solution, such as mazes.
- Finding biconnectivity in graphs and many more...

Breadth-First Search

- Uses queue (FIFO) to store nodes
- Used to go wide; checks if there's a connecting edge b/w 2 nodes. Basically it goes in layers of nodes away from a starting node; it goes as wide out as possible.
- Done iteratively, node by node

Algo (iterative):

1. Create a queue (linked list) to hold all the nodes; first push the starting node to it
2. Create a hash set to contain all the nodes that are already visited/seen
3. While loop for while the queue is not empty:
 - a. Pull first node out of the queue and check if it's in the seen set. If yes then pass to for loop
 - b. If no, then add it to the seen set, and 'process' it i.e., output it as a visited node
 - c. For loop, for each node adjacent to the current node:
 - i) Check if adjacent node is in seen set; if no then **push it to the queue**
 - ii) If it's already seen, then pass

Now, the queue has all the adjacent nodes; the while loop handles this then.

Main point of using queue here is that as we visit a vertex, we add all its immediate neighbours to the queue and then only we remove that current vertex from the start of the queue (i.e. it's 'explored') and then we go to the immediately succeeding nodes added to the queue which are those neighbours – since they're the next that have to be popped (FIFO), queues ensure that we go layer by layer.

```
# Perform BFS on the graph starting from vertex `v`
def BFS(graph, v, discovered):

    # create a queue for doing BFS
    q = deque()

    # mark the source vertex as discovered
    discovered[v] = True

    # enqueue source vertex
    q.append(v)

    # loop till queue is empty
    while q:

        # dequeue front node and print it
        v = q.popleft()
        print(v, end=' ')

        # do for every edge (v, u)
        for u in graph.adjList[v]:
            if not discovered[u]:
                # mark it as discovered and enqueue it
                discovered[u] = True
                q.append(u)
```

Recursive BFS:

```
# Perform BFS recursively on the graph
def recursiveBFS(graph, q, discovered):

    if not q:
        return

    # dequeue front node and print it
    v = q.popleft()
    print(v, end=' ')

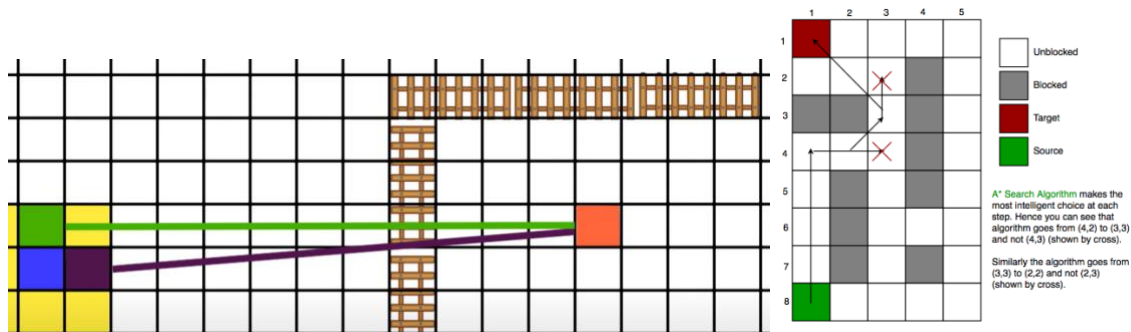
    # do for every edge (v, u)
    for u in graph.adjList[v]:
        if not discovered[u]:
            # mark it as discovered and enqueue it
            discovered[u] = True
            q.append(u)

    recursiveBFS(graph, q, discovered)
```

A-star algorithm

A path-finding algo, to find a path b/w 2 points in a graph.

Follows the most promising path – uses a '[heuristic](#)'/eval function to decide whether to take one node or another in its path, which calculated the estimated total cost of taking one of those paths. For eg.; a heuristic could be Euclidean distance b/w a node and the destination!



To make sure it doesn't visit the same node twice, it uses two lists: one to store the 'open' or unvisited nodes, and another to store 'closed' or visited nodes.

Further reading: <https://www.geeksforgeeks.org/a-search-algorithm/>

Algorithm :

1. Initialize the open list
2. Initialize the closed list
 - put the starting node on the open list (you can leave its f at zero; each node has an ' f ' = $g+h$; node w/ lower f is chosen for path)
3. while the open list is not empty
 - a) find the node with the least f on the open list, call it "q"
 - b) pop q off the open list
 - c) generate q's 8 (since 8 surrounding squares) successors and set their parents to q
 - d) for each successor
 - i) if successor is the goal, stop search
 - ii) else, compute both g and h for successor
 - successor. g = q. g + distance between successor and q
 - successor. h = distance from goal to successor (Manhattan. Diagonal, Euclidean)
 - successor. f = successor. g (**dist. Travelled**) + successor. h (**dist. Remaining**)
 - iii) if a node with the same position as successor is in the OPEN list which has a lower f than successor, skip this successor
 - iv) if a node with the same position as successor is in the CLOSED list which has a lower f than successor, skip this successor otherwise, add the node to the open list
 - end (for loop)
- e) push q on the closed list
- end (while loop)

Dijkstra's Algorithm

In Dijkstra's Algorithm, the goal is to find the shortest distance from a given source node to all other nodes in the graph. As the source node is the starting point, its distance is initialized to zero. From there, we iteratively pick the unprocessed node with the minimum distance from the source, this is where a min-heap (priority queue) or a set is typically used for efficiency. For each picked node u , we update the distance to its neighbors v using the formula: $\text{dist}[v] = \text{dist}[u] + \text{weight}[u][v]$, but only if this new path offers a shorter distance than the current known one. This process continues until all nodes have been processed.

Algo:

1. Set $\text{dist}[\text{source}] = 0$ and all other distances as infinity.
2. Push the source node into the min heap as a pair $\langle \text{distance}, \text{node} \rangle \rightarrow$ i.e., $\langle 0, \text{source} \rangle$.
3. Pop the top element (node with the smallest distance) from the min heap.
 1. For each adjacent neighbor of the current node:
 2. Calculate the distance using the formula:
 $\text{dist}[v] = \text{dist}[u] + \text{weight}[u][v]$
If this new distance is shorter than the current $\text{dist}[v]$, update it.
Push the updated pair $\langle \text{dist}[v], v \rangle$ into the min heap
4. **Repeat** step 3 until the min heap is empty.
5. Return the distance array, which holds the shortest distance from the source to all nodes.

```
# Returns shortest distances from src to all other vertices
def dijkstra(V, edges, src):
    # Create adjacency list
    adj = constructAdj(edges, V)
    # Create a priority queue to store vertices that
    # are being preprocessed.
    pq = []
    # Create a list for distances and initialize all
    # distances as infinite
    dist = [sys.maxsize] * V
    # Insert source itself in priority queue and initialize
    # its distance as 0.
    heapq.heappush(pq, [0, src])
    dist[src] = 0
    # Looping till priority queue becomes empty (or all
    # distances are not finalized)
    while pq:
        # The first vertex in pair is the minimum distance
        # vertex, extract it from priority queue.
        u = heapq.heappop(pq)[1]
        # Get all adjacent of u.
        for x in adj[u]:
            # Get vertex label and weight of current
            # adjacent of u.
            v, weight = x[0], x[1]
            # If there is shorter path to v through u.
            if dist[v] > dist[u] + weight:
                # Updating distance of v
                dist[v] = dist[u] + weight
                heapq.heappush(pq, [dist[v], v])
    # Return the shortest distance array
    return dist
```

Problems

[Number of Islands](#)

Given an $m \times n$ 2D binary grid `grid` which represents a map of '1's (land) and '0's (water), return *the number of islands*. An **island** is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

```
class Solution(object):
    def numIslands(self, grid):
        islands = 0
        visited = set()
        rows, cols = len(grid), len(grid[0])
        directions = [[-1, 0], [1, 0], [0, -1], [0, 1]]

        def bfs(row, col):
            visited.add((row, col))
            q = collections.deque() #initialised here since it's the start of a new island
            q.append((row, col))

            while q: #main thing!!! iterates over all of q's members
                r, c = q.popleft() #Gets next member of q in order
                for dr, dc in directions:
                    x, y = r + dr, c + dc
                    if x in range(rows) and y in range(cols) and grid[x][y] != '0' and (x,y) not in visited:
                        q.append((x,y))
                        visited.add((x,y))

            for i in range(rows):
                for j in range(cols):
                    if (i, j) not in visited and grid[i][j] != '0':
                        bfs(i, j) #explores the entire island and adds all of them to visited; so next time you wont go to the other members again.
                        islands += 1

        return islands
```

[Clone Graph](#)

Return a deep copy (clone) of the graph. For simplicity, each node's value is the same as the node's index (1-indexed). For example, the first node with `val == 1`, the second node with `val == 2`, and so on. The graph is represented in the test case using an adjacency list. An adjacency list is a collection of unordered lists used to represent a finite graph. Each list describes the set of neighbors of a node in the graph. The given node will always be the first node with `val = 1`. You must return the copy of the given node as a reference to the cloned graph.

```
"""
# Definition for a Node.
class Node(object):
    def __init__(self, val = 0, neighbors = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []
"""

class Solution(object):
    def cloneGraph(self, node):
        """
        :type node: Node
        :rtype: Node
        """

        #New graph to be returned will be a hashmap containing all the copy nodes!
        new = {} #map has old nodes as the properties, and new copy of that specific node as value.

        def copier(node): #Recursive DFS!
            # "processing"
            if node in new:
                return new[node] #the deepcopy version which has already been created in a previous iteration is returned.

            copy = Node(node.val) #making a new copy i.e., a new instance!
            new[node] = copy #MAP!!!!!! We map the og node to the new copy! Saving in the map helps for ensuring uniqueness

            for neighbor in node.neighbors: # "Exploration"! Recursive DFS essentially to get all neighbors
                neighborCopy = copier(neighbor) #returns a copy of the neighbor node!
                copy.neighbors.append(neighborCopy) #Adding the copied neighbor to the list!

            return copy #Finally, we return the new copy that has been created here.

        if node: #edge case; given node must not be null
            return copier(node) #since we're only asked to return a 'copy' of the given node.
```

Islands and Treasure

You're given a $m \times n \times n$ 2D grid initialized with these three possible values:

1. -1 - A water cell that *can not* be traversed.
2. 0 - A treasure chest.
3. INF - A land cell that *can* be traversed. We use the integer $2^{31} - 1 = 2147483647$ to represent INF.

Fill each land cell with the distance to its nearest treasure chest. If a land cell cannot reach a treasure chest then the value should remain INF. Assume the grid can only be traversed up, down, left, or right. Modify the grid **in-place**.

My solution with separate BFS from each treasure node: ($O(k \cdot m \cdot n)$; k = no. of treasure nodes)

```
class Solution:
    def islandsAndTreasure(self, grid: List[List[int]]) -> None:
        directions = [[-1, 0], [1, 0], [0, -1], [0, 1]]
        inf = 2**31 - 1
        #First, locate the treasure nodes, and from there do BFS till each inf and store the distances, making sure that -1 nodes aren't crossed.
        #Store the distance from each treasure node for every inf place in the grid and then replace it with the min of that, for each 0 node?
        rows, cols = len(grid), len(grid[0])

        def bfs(row, col): #will only be called for 0's
            visited = set() #initiating here, as we need a separate set for each 0 node.
            q = collections.deque()
            q.append((row, col, 0)) #Adds source node 0; third value is distance from this node

            while q:
                r, c, dist = q.popleft() #starting from source in first iteration
                for dr, dc in directions:
                    x = r + dr
                    y = c + dc
                    if x in range(rows) and y in range(cols) and grid[x][y] not in (-1, 0) and (x, y) not in visited:
                        visited.add((x, y))
                        newDist = dist + 1 #Main point: Since BFS goes layer by layer, we update each node in a layer with this
                        q.append((x, y, newDist))
                        if dist < grid[x][y]: #Since you're changing in place, don't check for grid[x][y] = inf specifically!
                            grid[x][y] = newDist

            for i in range(rows):
                for j in range(cols):
                    if grid[i][j] == 0:
                        bfs(i, j)
```

Same as BFS in no. of islands, except here it only starts from each treasure node and each visited set is initialized separately for it, and the distance calculation is done layer by layer and stored in the queue along with the coordinates of nodes to explore. Basically, this adds on the distance from the previous layer by one in each step made. It works, but not optimal!

Multi-Source BFS: Starts from multiple source nodes simultaneously. ($O(m \cdot n)$)

```
class Solution:
    def islandsAndTreasure(self, grid: List[List[int]]) -> None:
        rows, cols = len(grid), len(grid[0])
        directions = [[-1, 0], [1, 0], [0, -1], [0, 1]]
        queue = collections.deque()

        for i in range(rows):
            for j in range(cols):
                if grid[i][j] == 0:
                    queue.append((i, j)) # Add all treasure chests (0s) to queue as starting points

        while queue: # Multi-source BFS
            row, col = queue.popleft()
            for dr, dc in directions:
                new_row, new_col = row + dr, col + dc
                if (0 <= new_row < rows and 0 <= new_col < cols and grid[new_row][new_col] > grid[row][col] + 1):
                    grid[new_row][new_col] = grid[row][col] + 1
                    queue.append((new_row, new_col))
```

Initialization Phase:

- Add ALL source nodes to the queue simultaneously
- Mark all source nodes as visited/processed
- Set their initial distance/value as 0

Synchronized Exploration:

- Process nodes level by level (BFS property maintained)
- All nodes at distance 1 are processed before distance 2
- Each node naturally gets the shortest distance from its nearest source

Wave Propagation:

- Like ripples in water, the "wave" expands outward from all sources
- When waves from different sources meet, the optimal path has already been found

Distance Propagation Logic:

python

```
if grid[new_row][new_col] > grid[row][col] + 1:
```

```
    grid[new_row][new_col] = grid[row][col] + 1
```

This condition ensures we only update if we found a shorter path. Since BFS explores level by level, the first time we reach any cell is guaranteed to be via the shortest path¹.

No Separate Visited Array:

Unlike traditional BFS, we don't need a separate visited set because:

- We're modifying the grid in-place
- The condition `grid[cell] > current_distance + 1` naturally prevents revisiting with worse distances

[Rotting Oranges](#)

You are given an $m \times n$ grid where each cell can have one of three values:

- 0 representing an empty cell,
- 1 representing a fresh orange, or
- 2 representing a rotten orange.

Every minute, any fresh orange that is 4-directionally adjacent to a rotten orange becomes rotten. Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1.

My approach: same as prev qsn, I thought this was a case of counting total number of layers from each bfs and then adding it up:

```

class Solution(object):
    def orangesRotting(self, grid):
        #simple BFS where no. of layers of exploration
        directions = [[-1, 0], [1, 0], [0, -1], [0, 1]]
        visited, allOranges = set(), set()
        totalMins = 0
        rows, cols = len(grid), len(grid[0])

        def bfs(i, j):
            q = collections.deque()
            mins = 0
            q.append((i, j, mins)) #starting at 0 mins
            visited.add((i, j))

            while q:
                r, c, mins = q.popleft()
                for dr, dc in directions:
                    x, y = r + dr, c + dc
                    if x in range(rows) and y in range(cols) and (x,y) not in visited and grid[x][y] == 1:
                        visited.add((x, y))
                        q.append((x, y, mins + 1)) #mins +1 is the current layer number; gets added for all nodes in this iter!
                        grid[x][y] = 2
            return mins #since last layer's mins doesn't get updated in the if condition

        for i in range(rows):
            for j in range(cols):
                if grid[i][j] == 2 and (i,j) not in visited:
                    mins = bfs(i, j)
                    if mins >= totalMins:
                        totalMins = mins
                if grid[i][j] != 0:
                    allOranges.add((i, j))

        if visited == allOranges:
            return totalMins
        else:
            return -1

```

My method answers: "What's the maximum shortest path from any rotten orange to any fresh orange?" But the actual question is: "How long does it take for all rotten oranges to simultaneously spread until all reachable oranges are rotten?" I thought by taking the max exploration time (max since it would account for the other smaller oranges' exploration time) we'd get the answer, but in reality the rotten oranges act simultaneously so the final time will be lesser!

```

class Solution(object):
    def orangesRotting(self, grid):
        from collections import deque
        rows, cols = len(grid), len(grid[0])
        directions = [[-1, 0], [1, 0], [0, -1], [0, 1]]

        # Step 1: Initialize queue with ALL rotten oranges
        queue = deque()
        fresh_count, max_time = 0, 0
        for i in range(rows):
            for j in range(cols):
                if grid[i][j] == 2:
                    queue.append((i, j, 0))
                # Add ALL rotten oranges to queue with time=0
                elif grid[i][j] == 1:
                    fresh_count += 1

        # Step 2: Handle edge case
        if fresh_count == 0:
            return 0 # No fresh oranges to rot

        # Step 3: Multi-source BFS
        while queue:
            row, col, time = queue.popleft()
            # Process one orange from the front of queue
            max_time = max(max_time, time)
            # Track the maximum time we've seen
            for dr, dc in directions:
                new_row, new_col = row + dr, col + dc

                # Step 5: Boundary and validity checks
                if (0 <= new_row < rows and 0 <= new_col < cols and
                    grid[new_row][new_col] == 1):
                    # Step 6: Make orange rotten and add to queue
                    grid[new_row][new_col] = 2
                    fresh_count -= 1
                    # CRUCIAL: Add with time + 1
                    queue.append((new_row, new_col, time + 1))

        # Step 7: Final check
        return max_time if fresh_count == 0 else -1

```

This is why we need **Multi-source BFS**: here we initiate the queue only once at the start as all sources get handled simultaneously, not sequentially – the 'simultaneuosuness' is handled by coords having the same minute value, and it is no longer sequential because previously each source has its own queue that was processed sequentially, but here there's only one global queue for all sources. First, find all 2's (sources) and count 1's.

Then, normal BFS occurs; layer==minute; All minute-0 oranges are processed first then all minute-1 oranges, then minute-2.. ➔ correct temporal order!

```

Time 0: Queue = [(0,0,0), (0,4,0)]
Grid: 2 1 1 2
      1 1 0 1
      1 1 1 1

Time 1: Queue = [(0,1,1), (1,0,1), (0,3,1), (1,4,1)]
Grid: 2 2 1 2
      2 1 0 2
      1 1 1 1

Time 2: Queue = [(0,2,2), (1,1,2), (2,0,2), (1,3,2), (2,4,2)]
Grid: 2 2 2 2
      2 2 0 2
      2 1 1 2

Time 3: Queue = [(2,1,3), (2,2,3), (2,3,3), (1,2,3)]
Grid: 2 2 2 2
      2 2 2 2
      2 2 2 2

```

Pacific Atlantic Water Flow

There is an $m \times n$ rectangular island that borders both the Pacific Ocean and Atlantic Ocean. The Pacific Ocean touches the island's left and top edges, and the Atlantic Ocean touches the island's right and bottom edges. The island is partitioned into a grid of square cells. You are given an $m \times n$ integer matrix `heights` where `heights[r][c]` represents the height above sea level of the cell at coordinate (r, c) . The island receives a lot of rain, and the rain water can flow to neighboring cells directly north, south, east, and west if the neighboring cell's height is less than or equal to the current cell's height. Water can flow from any cell adjacent to an ocean into the ocean.

Pacific Ocean				
1	2	2	3	5
3	2	3	4	4
2	4	5	3	1
6	7	1	4	5
5	1	1	2	4
Atlantic Ocean				

Return a 2D list of grid coordinates result where $\text{result}[i] = [r_i, c_i]$ denotes that rain water can flow from cell (r_i, c_i) to both the Pacific and Atlantic oceans.

My approach: Get the coords of pacific and Atlantic neighbouring cells, and then do DFS from each inner cell with the condition that the next node's height is lesser.

Got stuck: was trying to get the bordering cells' coords and storing them in arrays!

Correct way: Store them in a set (visited set!), and instead do DFS from every border cell to see what all inner points can be reached; then select the inner points reach by both types of border cells.

Just start from 1st and last rows for the DFS searches and then 1st and last cols separately for pac/atl.

This time with DFS function we also add the specific ocean type's visited set (pac/atl; no visited set initiation inside dfs) and check height of the current node you're visiting/starting from, to use in the check condition ($\leq$). I've done in both recursive and iterative DFS, understand properly:

```

class Solution(object):
    def pacificAtlantic(self, heights):
        #applying DFS to check if a path exists to both pacific & atlantic oceans; with condition that next node's height is lesser
        rows, cols = len(heights), len(heights[0])
        directions = [[-1, 0], [1, 0], [0, 1], [0, -1]]
        pac, atl = set(), set()

        def dfs(row, col, visited): #r, c of current node
            visited.add((row,col)) #add current node
            """#iterative DFS:
            s = [] #stack for dfs from this specific node!
            s.append((row,col))
            while s:
                r, c = s.pop()
                for dr, dc in directions:
                    x, y = r + dr, c + dc
                    if x in range(rows) and y in range(cols) and (x, y) not in visited and heights[r][c] <= heights[x][y]:
                        #going up; need more height or equal height, that works too! dont forget.
                        s.append((x,y))#stack/queue here to keep track of nodes to do in this path specifically?
                        visited.add((x,y))
            """
            """ #recursive dfs:
            for dr, dc in directions:
                x, y = row + dr, col + dc #remember, we're comparing the current node's height against the other's!
                if x in range(rows) and y in range(cols) and (x, y) not in visited and heights[row][col] <= heights[x][y]:
                    dfs(x, y, visited)
            """

        for i in range(cols):
            dfs(0, i, pac)
            dfs(rows - 1, i, atl)

        for j in range(rows):
            dfs(j, 0, pac)
            dfs(j, cols - 1, atl)

        res = []
        for a in range(rows):
            for b in range(cols):
                if (a, b) in pac and (a, b) in atl:
                    res.append([a, b])

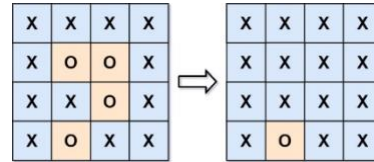
        return res

```

Surrounded Regions

You are given an $m \times n$ matrix board containing letters 'X' and 'O', capture regions that are **surrounded**:

- **Connect**: A cell is connected to adjacent cells horizontally or vertically.
- **Region**: To form a region **connect every** 'O' cell.
- **Surround**: The region is surrounded with 'X' cells if you can **connect the region** with 'X' cells and none of the region cells are on the edge of the board.



To capture a **surrounded region**, replace all 'O's with 'X's **in-place** within the original board. You do not need to return anything.

My initial thought: Do DFS from each O node, check for border cases, mark them as X

Correct way: **Just do DFS from the edge O's and then exclude the O's in this path while turning everything else to X! Classic DFS.**

Also for recursive dfs, remember that the if condition must act on all the 4 direction's nodes' coords first before passing it to next dfs iteration/adding to stack!!

```
class Solution(object):
    def solve(self, board):
        rows, cols = len(board), len(board[0])
        directions = [[-1, 0], [1, 0], [0, -1], [0, 1]]
        toExclude = set()

        def dfs(r, c):
            toExclude.add((r,c))
            for dr, dc in directions:
                x, y = r + dr, c + dc
                if (x in range(rows) and y in range(cols) and
                    (x, y) not in toExclude and board[x][y] == 'O'):
                    dfs(x, y)

        for i in range(rows): #left & rightmost columns
            if board[i][0] == 'O':
                dfs(i, 0)
            if board[i][cols-1] == 'O':
                dfs(i, cols-1)

        for j in range(cols): #top & bottom-most rows
            if board[0][j] == 'O':
                dfs(0, j)
            if board[rows-1][j] == 'O':
                dfs(rows-1, j)

        for a in range(rows):
            for b in range(cols):
                if (a, b) not in toExclude and board[a][b] == 'O':
                    board[a][b] = 'X'
```

Course Schedule

(IMPORTANT)

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [a_i, b_i] indicates that you **must** take course b_i first if you want to take course a_i. For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1. Return true if you can finish all courses, else, false.

Example 1:

Input: numCourses = 2, prerequisites = [[1,0]]

Output: true

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0. So it is possible.

Example 2:

Input: numCourses = 2, prerequisites = [[1,0],[0,1]]

Output: false

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

My approach: try to form all the possible paths that can be made, then see if any path length == no. of courses. I tried to make a map of prereq → course that had arrays of succeeding courses that can be taken next for each prereq course, then I thought that using a dfs algo to iterate thru this and find the longest path possible would suffice, if it's greater than the number of courses.

➔ Read qsn wrong, total number of courses == distinct no. of courses in the input; u just need to make sure if all of these can be done sequentially i.e., linear graph!!!

Mistake: didn't account for cycles!! Each prereq-course relation is an edge in a graph. However, we can have a bidirectional edge when two courses are prereq's of each other; now it's impossible to do these courses. Consider $c1 \rightarrow c2 \rightarrow c3 \rightarrow c1$; this is a 3-member cycle. We need to make sure there are no such cycles in the graph!

First, as I did in my implementation, make a map of the courses as keys & a list of courses that are prereqs for it as values, so that u climb in an 'ascending order'.

We use a visit set to store the linear path we're going along with DFS, so as to ensure we're not looping back which would result in the node already being in the set.

Then, we make a recursive DFS with 2 basecases:

- 1: If the course (node) is already added to visit set
➔ loop i.e., cycle detected; no way to complete all
- 2: If the course has an empty list, it has no higher prereqs ➔ can return true; end is reached.

Next, we add it to the current path done so far. Next, the for loop goes through each of the current course's prereqs and goes all the way till the end of the paths starting from each of them – if we end up finding any one course from these paths that can't be completed (due to a cycle), then return False for the whole

Now once this for loop is done and we don't get False, we know that the current course is good and can be done, as it doesn't have any cycles in its prereq path. However, if it remained in the visited set, we could trip the first if-clause in the dfs function if we revisit it and return False. That's what we don't want to do, because we just calculated that crs has no cycles. So we remove it from visited so that other paths can successfully revisit it (is this backtracking?). Basically we can visit the node twice without it being a cycle due to it terminating multiple paths.

Now we know that this course can be successfully done along with all of its prerequisites. So, we mark its prereqs as an empty list so that for the next crs we're running dfs for, it will see this empty list and fulfil the second if condition and return true from that point, since all the prereqs from then on can be done successfully. MAKES SENSE, PRETTY SMART FOR SAVING TIME!

Then finally, we do this dfs with each of the courses as a starting point; starting from anyone should make all of them reachable.

1-350, 990-2801

<https://internship.iitm.ac.in/downloadstudentresume/2801/1>

```
class Solution:
    def canFinish(self, numCourses, prerequisites):
        # Map each course to its prerequisites
        preMap = {i: [] for i in range(numCourses)}
        for crs, pre in prerequisites:
            preMap[crs].append(pre)

        # Store all courses along the current DFS path
        visiting = set()

        def dfs(crs):
            if crs in visiting:
                # Cycle detected
                return False
            if preMap[crs] == []:
                return True

            visiting.add(crs)
            for pre in preMap[crs]:
                if not dfs(pre):
                    return False
            visiting.remove(crs)
            preMap[crs] = []
            return True

        for c in range(numCourses):
            if not dfs(c):
                return False
        return True
```

Course Schedule II

You are given an array prerequisites where prerequisites[i] = [a, b] indicates that you **must** take course b first if you want to take course a. For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1. There are a total of numCourses courses you are required to take, labeled from 0 to numCourses - 1. Return a valid ordering of courses you can take to finish all courses. If there are many valid answers, return **any** of them. If it's not possible to finish all courses, return an **empty array**.

Example 1: Input: numCourses = 3, prerequisites = [[1,0]]; Output: [0,1,2]

Explanation: We must ensure that course 0 is taken before course 1.

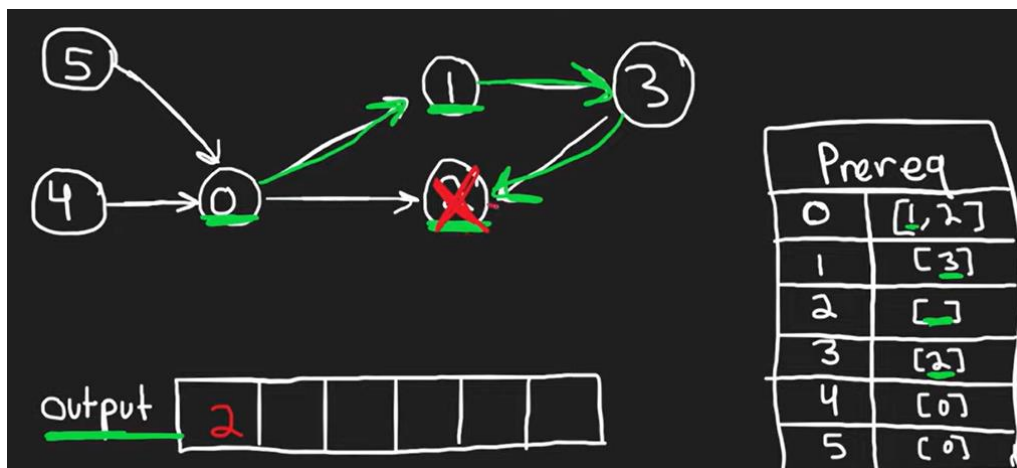
Example 2: Input: numCourses = 3, prerequisites = [[0,1],[1,2],[2,0]]; Output: []

Explanation: It's impossible to finish all courses.

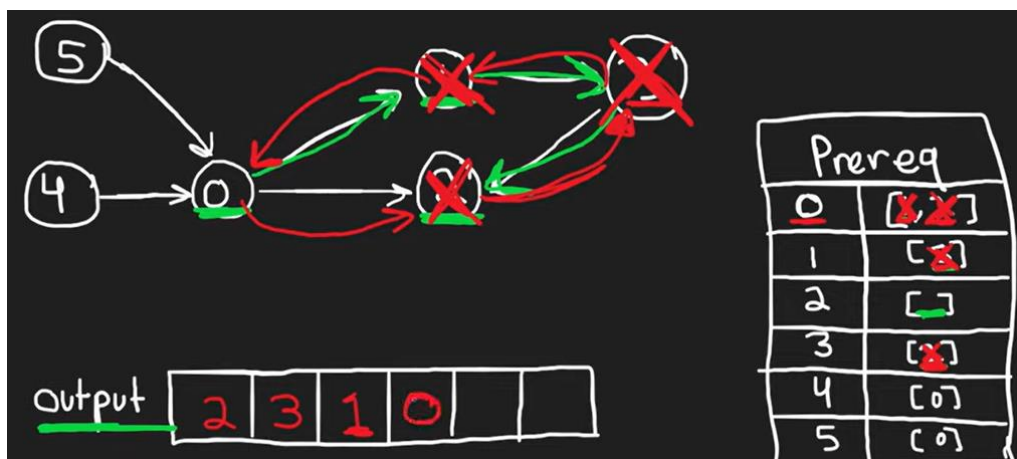
This is very similar to course schedule 1, except here we need to return an ordering of the courses such that all courses are done!

To solve this, we use **Topological Sort**!

1. Similar to CS1, we run DFS on every given course (node)



Here, starting with DFS on 0, we go down the path from its first prereq: 1; this leads us down till 2 which has no prereqs. We add 2 to our output path and then remove it from our current path as we no longer have to ever visit it again.



Next, we go back to node 3, see that it has no other prereqs (in prereq map array for it) and then go to 1 and then finally back to 0, where we see that its other prereq 2 is already in the output path and hence already visited. Each of them get added to the output path as they have no more prereqs!

Since 2,3,1,0 are marked as visited, we don't have to do DFS on them anymore.

Now once 0 is done, we can do 4 and 5 in any order i.e., 4 then 5 or 5 then 4. We do DFS on 4, and see that it has 0 as a prereq, and 0 is already visited, so now 4 is visited and added to the output. The same is done for 5. Final order: 2, 3, 1, 0, 4, 5

Remember: here, a course can be in 3 possible states: visited (added to final output path already), visiting (in current path of traversal; used for cycle detection) and unvisited (yet to visit).

This is the topological sort of this graph! Time complexity: $O(E+V)$

If a cycle is detected, we have to return an empty list. For this, we'll use a hash set to store the current path being traversed; if we visit a node that's already in it then we know a cycle is there.

```
class Solution:
    def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]:
        preMap = {i: [] for i in range(numCourses)}
        for crs, pre in prerequisites:
            preMap[crs].append(pre)

        visiting = set() #Store all courses along the current DFS path
        visited = set() #Tracks which courses have been completely processed (meaning we've explored all their prerequisites and confirmed no cycles)
        output = [] #Maintains the correct ordering of courses based on when they finish processing

        def dfs(crs):
            if crs in visiting: #Cycle detected
                return False #terminate DFS here
            if crs in visited:
                return True #Don't need to visit this again

            visiting.add(crs) #Add to current path
            for pre in preMap[crs]: #goes down all its prereqs
                if not dfs(pre): #For each node that doesn't end up in a cycle, it gets added to visited and removed from visiting, in bottom->up!
                    return False #If cycle, end DFS

            #At this point, no cycles are found in the path starting from here
            #Now, it's no longer in the current path that we're going in, we can add it to visited too
            visiting.remove(crs)
            visited.add(crs)
            output.append(crs)
            return True

        for crs in range(numCourses):
            if not dfs(crs): #Cycle detected
                return []
        return output
```

Understand this properly!!!

Why separate output and visited? The key insight is that **sets don't preserve order**, but we need the courses in a specific sequence. The output list captures the **reverse topological order** - courses are added when they have no remaining unprocessed prerequisites.

Graph Valid Tree

Given n nodes labeled from 0 to $n - 1$ and a list of **undirected** edges (each edge is a pair of nodes), write a function to check whether these edges make up a valid tree.

Example 1: Input: $n = 5$, edges = $[[0, 1], [0, 2], [0, 3], [1, 4]]$; Output: true

Example 2: Input: $n = 5$, edges = $[[0, 1], [1, 2], [2, 3], [1, 3], [1, 4]]$; Output: false

Similar to prev qsn except here the edges are bidirectional; so the dfs function has to be changed to know which is the previous node and which is the current node, so that you explore the next node in line only if it's not equal to the previous node. Along with this, standard cycle checking. Revise!

```
class Solution:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        #see if no cycle is there, and if all edges are connected (len(tree) == no. of nodes)
        edgeMap = {i: [] for i in range(n)}
        for node1, node2 in edges:
            edgeMap[node1].append(node2) #one node can have several branches!
            edgeMap[node2].append(node1) #bidirectional connections
        tree = set()

        #due to the bidirectional connections stored, we need dfs to track prev node too
        #thus base case is when we reach a node which has only one connection (to prev node)
        def dfs(node, prevNode):
            if node in tree:
                return False
            tree.add(node)
            for nextNode in edgeMap[node]:
                if nextNode != prevNode and not dfs(nextNode, node):
                    return False
            return True

        #since we're given no. of nodes, we assume we start from node 0 always?
        return dfs(0, -1) and len(tree) == n #-1 b4 0 since it's not in given list anyways
```

Number of Connected Components in an Undirected Graph

There is an undirected graph with n nodes. There is also an edges array, where edges[i] = [a, b] means that there is an edge between node a and node b in the graph. The nodes are numbered from 0 to $n - 1$. Return the total number of connected components in that graph.

Example 1: Input: $n=3$, edges= $[[0,1], [0,2]]$; Output: 1

Example 2: Input: $n=6$, edges= $[[0,1], [1,2], [2,3], [4,5]]$; Output: 2

No need to check for cycles; just keep adding nodes to visited set as the first path starts and keep endcase as stopping when there are no more unvisited neighbours; then go thru non-visited nodes and start a new dfs for them → new path; then return total number of paths forms.

```
class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        #no need to check for cycles here?
        #Once one exploration is done, start from another unconnected node
        #So as to find the total distinct no. of paths ==> start new path set
        edgeMap = {i: [] for i in range(n)}
        for node1, node2 in edges:
            edgeMap[node1].append(node2)
            edgeMap[node2].append(node1) # Important: undirected graph

        visited = set()

        def dfs(node):
            if node in visited:
                return #stops the search
            visited.add(node)
            for nextNode in edgeMap[node]:
                dfs(nextNode) #continues till end of this path

        count = 0
        for node in range(n):
            if node not in visited: #nodes in one path are already added to visited!
                dfs(node) #starts a new path!
                count += 1
```

Alien Dictionary (Advanced Graphs!)

There is a foreign language which uses the latin alphabet, but the order among letters is *not* "a", "b", "c" ... "z" as in English. You receive a list of *non-empty* strings words from the dictionary, where the words are **sorted lexicographically** based on the rules of this new language. Derive the order of letters in this language. If the order is invalid, return an empty string.

A string a is lexicographically smaller than a string b if either of the following is true:

- The first letter where they differ is smaller in a than in b.
- There is no index i such that $a[i] \neq b[i]$ and $a.length < b.length$.

Example 1: Input: ["z", "o"], Output: "zo";

Explanation: From "z" and "o", we know 'z' < 'o', so return "zo".

Example 2: Input: ["hrn", "hrf", "er", "enn", "rfnn"], Output: "hernf";

Explanation:

from "hrn" and "hrf", we know 'n' < 'f'

from "hrf" and "er", we know 'h' < 'e'

from "er" and "enn", we know get 'r' < 'n'

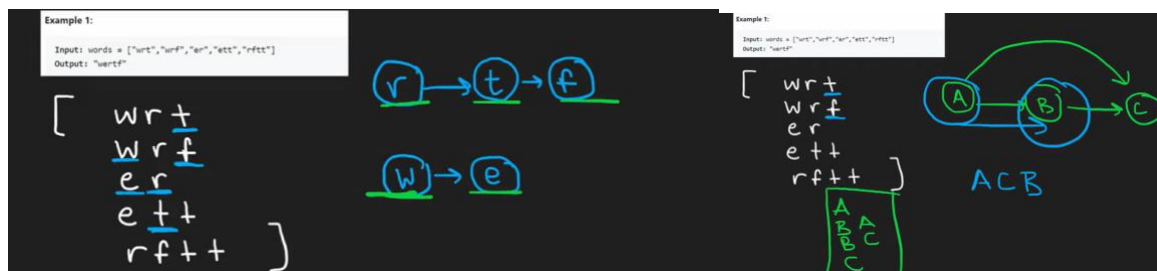
from "enn" and "rfnn" we know 'e' < 'r'

so one possible solution is "hernf"

This is a problem on a DAG (Directed Acyclic Graph) where all the edges have one direction and there are no cycles. We have to use topological graph for it.

First idea was correct: create edges from higher to lower letters, by iterating over the characters of every successive pair of words and saving the edge if their corresponding characters are unequal.

Now, it's possible that the resulting edges might have some lacking connection – in this case, the final ans can be any combination of the continuous edge paths that we can find, as we just have to make sure that each formed path's members are arranged in the correct order. (left image)



However we cant just do a direct dfs along the edges too – consider the right image; here the correct ordering is obtained if we go down the blue bath with dfs to get ABC, but since there's a second edge from A to C, dfs might go down that first and give the incorrect order of ACB.

For this, we use **Post-order DFS**: process all the descendants first, adding the childless ones first, before then adding the starting node to the visited set; then reverse of visited set is the order.

Along with this, we need to do loop detection as well, as done in previous questions. Note that since we're using post-order dfs which goes through the childless descendants first, these would be already visited, and then when we're going to another descendant that's above the childless descendant, that childless descendant will be marked as already visited – we cant use this as our cycle checking condition!

So we need to track 2 things:

1. If a node has already been visited, and 2. If a node is in our current path

For this, we'll use a dictionary. When we first visit a node, we'll put a False value for that node in the dictionary. Once it's actually in the current path that we're traversing, we'll set its value to True. If this value is True, then it can be used for the cycle-detection case.

Finally, we do this dfs on each character – it works if we start on any character? Map it out and see.

```
class Solution:
    def foreignDictionary(self, words: List[str]) -> str:
        #from the given list we can make an adjacency list; with edges from bigger to smaller letter?
        #go two words at a time and compare their non-equal letters; then save the edges
        edges = {char : set() for word in words for char in word} #creating a set for all chars given
        for i in range(len(words)-1):
            w1, w2 = words[i], words[i+1]
            minLen = min(len(w1), len(w2))
            if len(w1) > len(w2) and w1[:minLen] == w2[:minLen]:
                return "" #invalid contradiction; impossible order; no solution.
            for j in range(minLen):
                if w1[j] != w2[j]: #first unequal char; save the edge
                    edges[w1[j]].add(w2[j])
                    break #move to next pair of words

        visited = {} #False -> visited; True -> in current path
        result = []
        def dfs(letter):
            if letter in visited:
                return visited[letter] #if False, continue dfs. If True -> loop! -> exit
            visited[letter] = True #currently in the path
            for nextletter in edges[letter]:
                if dfs(nextletter): #if we get True -> loop!
                    return
            visited[letter] = False #if the above code works, then there's a linear path formed
            #same as visiting.remove(crs) in course scheduler - bit confused about this!
            result.append(letter)

        for char in edges:
            if dfs(char):
                return "" #we get True -> loop!

        result.reverse()
        return "".join(result)
```

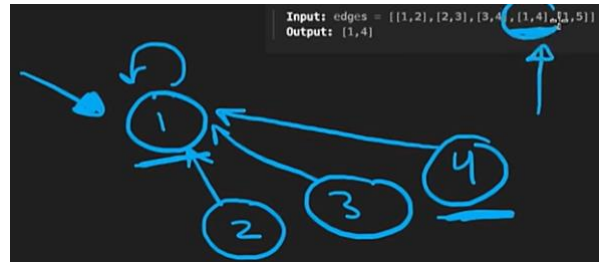
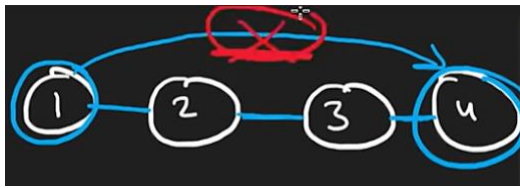
[Redundant Connection](#) (Google Dev Guide)

You are given a connected **undirected graph** with n nodes labeled from 1 to n . Initially, it contained no cycles and consisted of $n-1$ edges. We have now added one additional edge to the graph. The edge has two **different** vertices chosen from 1 to n , and was not an edge that previously existed in the graph. The graph is represented as an array `edges` of length n where `edges[i] = [ai, bi]` represents an edge between nodes ai and bi in the graph. Return an edge that can be removed so that the graph is still a connected non-cyclical graph. If there are multiple answers, return the edge that appears last in the input edges.

Example 1: Input: `edges = [[1,2],[1,3],[3,4],[2,4]]`; Output: `[2,4]`

Example 2: Input: `edges = [[1,2],[1,3],[1,4],[3,4],[4,5]]`; Output: `[3,4]`

Here, to detect cycles, we use **Union Find/Disjoint set!**



Union Find can tell us that there's already a path existing from 1 to 4, and when we introduce a new connection from 1 to 4, we can identify that it's a cycle. It creates a compressed path where all the nodes that belong to a path point directly to a root node; thus just 2 levels: root & members!

When the next 1 to 4 edge is added, we see that 4 already points to the root node of 1; hence a cycle has been detected. Time complexity: $O(V+E)$

Understand this code properly!

First initiate a list where each element corresponds to the root node for that index's node; initially all nodes point to themselves and are not in any path.

Then, make a recursive find function that traverses through the above list and gets the root node for a given node and stores it against its index in that list.

Next, the union finds the root node of 2 given elements; if they're the same then the 2 nodes are in the same path already and it returns False, as a new connection between n1 and n2 is being added and therefore a cycle is being made. Else, we set the parent of the second node as the first node, hence making a union set i.e., adding them to the same path.

Finally we process the edges in input order; the first edge that creates a cycle is the redundant one, and this gives us the "last occurring" redundant edge.

```
class Solution(object):
    def findRedundantConnection(self, edges):
        N = len(edges) #Since tree with N nodes -> N-1 edges; and here one edge is extra
        parent = [i for i in range(N+1)] #ith element = parent of ith node
        #initially, each node is the parent of itself => [0, 1, 2, 3, ...] i.e. all are roots

        def find(n): #returns the parent/root node of given node; makes all path nodes point to root
            if n != parent[n]: #this means n is not its own parent i.e. not a root
                parent[n] = find(parent[n]) #recursively gets the root node of n
            return parent[n]

        def union(n1, n2): #returns True if n1, n2 are not already connected
            p1, p2 = find(n1), find(n2)
            if p1 == p2:
                return False #Same root node; thus they share an existing path
            parent[p1] = p2 #else, this puts them on the same path
            return True

        for n1, n2 in edges:
            if not union(n1, n2):
                return [n1, n2] #this was the redundant edge; as n1 & n2 are connected

        return []
```

Word Ladder

You are given two words, `beginWord` and `endWord`, and also a list of words `wordList`. All of the given words are of the same length, consisting of lowercase English letters, and are all distinct. Your goal is to transform `beginWord` into `endWord` by following the rules: You may transform `beginWord` to any word within `wordList`, provided that at exactly one position the words have a different character, and the rest of the positions have the same characters. You may repeat the previous step with the new word that you obtain, and you may do this as many times as needed. Return the **minimum number of words within the transformation sequence** needed to obtain the `endWord`, or 0 if no such sequence exists. **Examples:**

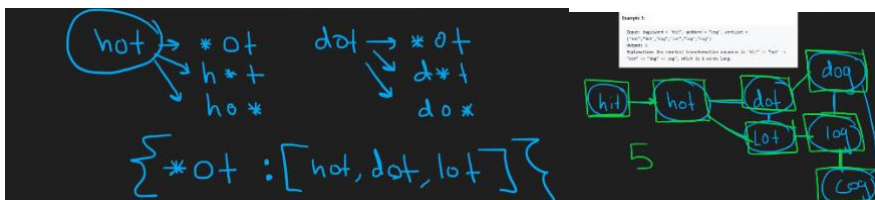
Input: `beginWord = "cat", endWord = "sag", wordList = ["bat", "bag", "sag", "dag", "dot"]`; Output: 4

Explanation: The transformation sequence is "cat" -> "bat" -> "bag" -> "sag".

Input: `beginWord = "cat", endWord = "sag", wordList = ["bat", "bag", "sat", "dag", "dot"]`; Output: 0

Explanation: There is no possible transformation sequence from "cat" to "sag"; is not in the wordList.

The way to make an adjacency list efficiently is by taking all the words and storing all the possible patterns/string templates with one character removed and seeing if other words have that pattern.



Store in a hash map; key is the pattern and value is the array with words having that pattern. To get all neighbours of a given word, find all patterns generatable from it and then we use this map; for eg., we get 3 different neighbour arrays for hot (`_ot`, `h_t` and `ho_` as keys!). Complexity: $O(nm^2)$; n = no. of words and m = len of word. To find the shortest path, we use BFS here!

```
class Solution:
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str]) -> int:
        #Make a path - with each next word having only one char different

        if endWord not in wordList:
            return 0

        neighbours = collections.defaultdict(list) #default value of every key is a list here. Convenient!
        wordList.append(beginWord) #Has to be included!

        for word in wordList:
            for j in range(len(word)): #form all patterns
                pattern = word[:j] + "*" + word[j+1:]
                neighbours[pattern].append(word) #if the same pattern is found in another word, it gets added to the same key!

        visited = set([beginWord]) #already includes beginWord?
        q = collections.deque([beginWord]) #goes layer by layer in BFS
        pathLen = 1
        while q: #if loop runs successfully, we return the pathLen. Else, return 0 - no path!
            for i in range(len(q)):
                currentWord = q.popleft()
                if currentWord == endWord:
                    return pathLen #return once we reach the end!!
                for j in range(len(word)): #form all patterns
                    pattern = currentWord[:j] + "*" + currentWord[j+1:]
                    for nextWord in neighbours[pattern]:
                        if nextWord not in visited:
                            visited.add(nextWord)
                            q.append(nextWord)
                pathLen += 1 #once node/layer is processed, add it to total length!

        return 0 #if we dont reach the endWord from above while loop
```

Backtracking

Recursively/iteratively build towards solns w/ temp ones, and abandon the temp one if this doesn't reach desired final soln.

Problems

Combination Sum

You are given an array of **distinct** integers `nums` and a target integer `target`. Your task is to return a list of all **unique combinations** of `nums` where the chosen numbers sum to `target`.

The **same** number may be chosen from `nums` an **unlimited number of times**. Two combinations are the same if the frequency of each of the chosen numbers is the same, otherwise they are different.

You may return the combinations in **any order** and the order of the numbers in each combination can be in **any order**.

Example 1: Input: `nums = [2,5,6,9]` `target = 9`;
Output: `[[2,2,5],[9]]`

Explanation:

$2 + 2 + 5 = 9$. We use 2 twice, and 5 once. $9 = 9$.

Example 2: Input: `nums = [3,4,5]` `target = 16`;
Output: `[[3,3,3,3,4],[3,3,5,5],[4,4,4,4],[3,4,4,5]]`

Example 3: Input: `nums = [3]` `target = 5`; Output: `[]`

Wrote a recursive function to keep searching for elements lesser than the remainder from target and adding those to a combinations array. Correct idea; some mistakes in implementation! Understand!

```
Initial call: getCombination(0, [], 9)
combination = []

| Try nums[0]=2: combination.append(2)
| combination = [2]
| |
| | Try nums[0]=2 again: combination.append(2)
| | combination = [2,2]
| | |
| | | Try nums[1]=5: combination.append(5)
| | | combination = [2,2,5] ← remainder=0, FOUND SOLUTION!
| | | combinations.append([2,2,5]) ← Copy saved
| | | combination.pop() ← Remove 5
| | | combination = [2,2]
| | |
| | | Try nums[2]=6: 6 > remainder(5), skip
| | | Try nums[3]=9: 9 > remainder(5), skip
| |
| | combination.pop() ← Remove second 2
| | combination = [2]
| |
| | Try nums[1]=5: 5 > remainder(7), skip
| | Try nums[2]=6: 6 > remainder(7), skip
|
| combination.pop() ← Remove first 2
| combination = []
|
| Try nums[1]=5: combination.append(5)
| combination = [5]
| | (no valid solutions found)
| | combination.pop() ← Remove 5
| | combination = []
|
| Try nums[2]=6: combination.append(6)
| combination = [6]
| | (no valid solutions found)
| | combination.pop() ← Remove 6
| | combination = []
|
| Try nums[3]=9: combination.append(9)
| combination = [9] ← remainder=0, FOUND SOLUTION!
| combinations.append([9]) ← Copy saved
| combination.pop() ← Remove 9
| combination = []
```

```
class Solution:
    def combinationSum(self, nums: List[int], target: int) -> List[List[int]]:
        #iterate thru list: do target - element; check if remainder in list;
        #while remainder >=0 element keep going recursively, else abandon approach.

        combinations = []
        nums.sort()

        def getCombinations(combination, remainder):
            if remainder == 0 and sorted(combination) not in combinations:
                combinations.append(sorted(combination[:]))
            #appending copy since if we append combination itself, it's a direct reference and that becomes [] from popping
            #No need to create a new combination array - it gets popped down to [] the deeper the recursion goes!
            for element in nums:
                if element > remainder:
                    break #Abandoning this combination!
                combination.append(element)
                newRemainder = remainder - element
                getCombination(combination, newRemainder)
                combination.pop()
                #^^IMPORTANT - MISSED THIS! Need to pop so combination can go to next element iteration

        getCombinations([], target)

        return combinations
```

Word Search

Given a 2-D grid of characters board and a string word, return true if the word is present in the grid, otherwise return false.

For the word to be present it must be possible to form it with a path in the board with horizontally or vertically neighboring cells. The same cell may not be used more than once in a word.

Example 1:

Input:

board = [["A","B","C","D"], ["S","A","A","T"], ["A","C","A","E"]], word = "CAT"; Output: true

A	B	C	D
S	A	A	T
A	C	A	E

Example 2:

Input:

board = [["A","B","C","D"], ["S","A","A","T"], ["A","C","A","E"]], word = "BAT" Output: false

A	B	C	D
S	A	A	T
A	C	A	E

Solved it using simple DFS. Important to understand: copying the path!

```
class Solution:
    def exist(self, board: List[List[str]], word: str) -> bool:
        directions = [[-1, 0], [1, 0], [0, 1], [0, -1]]
        rows, cols = len(board), len(board[0])

        def dfs(r, c):
            s = [(r, c, 0, {(r,c)})] #stack with coords, current letter index and current path
            while s:
                r, c, currentLetter, visited = s.pop()
                if currentLetter == len(word) - 1:
                    return True #Base case: found complete word
                for dr, dc in directions:
                    x, y = r + dr, c + dc
                    if (x in range(rows) and y in range(cols) #remember, we're onto the next character now!
                        and board[x][y] == word[currentLetter + 1] and (x,y) not in visited):
                        path = visited.copy() #multiple paths exist! Without this, paths would contaminate others' sets
                        path.add((x,y)) #IMPORTANT: COPYING! Ensures each path can explore all possibilities
                        s.append((x, y, currentLetter + 1, path))
            return False #if above code executes fully and no True is returned i.e., no end met

        for i in range(rows):
            for j in range(cols):
                if board[i][j] == word[0] and dfs(i,j): #start of word == start of path
                    return True

        return False
```

Greedy

<https://www.hackerearth.com/practice/algorithms/greedy/basics-of-greedy-algorithms/tutorial/>

Basically making an optimisation rule locally hoping that it works for all cases?

Eg.: You have a fixed amount of time to do n tasks; each task takes a diff amount of time and each task has a different priority of importance. You want to do as many tasks as possible but also the important ones first. If doing these 2 independently, then u can just sort the array with tasks based on priority or time taken respectively. But to account for both, you have to ensure that the tasks you do have the most optimal priority and time taken → rank based on priority/time taken ratio!

```
Algorithm (P, T, N)
{
    let S be an array of pairs ( C++ STL pair ) to store the scores and their indices
    , C be the completion times and F be the objective function
    for i from 1 to N:
        S[i] = ( P[i] / T[i], i )          // Algorithm #2
    sort(S)
    C = 0
    F = 0
    for i from 1 to N:                    // Greedily choose the best choice
        C = C + T[S[i].second]
        F = F + P[S[i].second]*C
    return F
}
```

Problems

Maximum Subarray

https://en.wikipedia.org/wiki/Maximum_subarray_problem

Given an array of integers `nums`, find the subarray with the largest sum and return the sum.

A **subarray** is a contiguous non-empty sequence of elements within an array.

Example 1: Input: `nums = [2,-3,4,-2,2,1,-1,4]`; Output: 8

Explanation: The subarray `[4,-2,2,1,-1,4]` has the largest sum 8.

We can do it in a prefix sum manner, with two pointers: one indicating the left end of the subarray, and another for the right – kind of like a sliding window! Increment the right pointer one by one and see the sum of elements from left pointer till right pointer; if the sum is negative i.e., we have a negative prefix, then we shift the left pointer ahead so as to remove the negative contribution. Now instead of doing it with 2 pointers//sliding window we can represent the entire subarray being processed just as its sum instead for efficiency! → KADANE'S ALGORITHM! → $O(n)$

```
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        maxSubarray = nums[0] #starts with first element
        currentSum = 0 #this is the sum of the 'sliding window'

        for n in nums:
            if currentSum < 0:
                currentSum = 0 #restarting the sum
            currentSum += n
            maxSubarray = max(maxSubarray, currentSum)

        return maxSubarray
```

Jump Game

You are given an integer array `nums` where each element `nums[i]` indicates your maximum jump length at that position. Return `true` if you can reach the last index starting from index 0, or `false` otherwise.

Example 1: Input: `nums = [1,2,0,1,0]`; Output: `true`

Explanation: First jump from index 0 to 1, then from index 1 to 3, and lastly from index 3 to 4.

Example 2: Input: `nums = [1,2,1,0,1]`; Output: `false`

A greedy approach is to start from the end instead. If the last element can be jumped to from the second last element (i.e $l[n-2] \geq 1$) then we can shift our goalpost/destination from being the end to being the second last element instead since we know the end can be reached from there. Then, just keep shifting the goalpost leftwards till the starting element by checking if the immediately preceding element's value is greater than or equal to 1, and if not, then searching till the start for an element that can jump to it (so as to handle 0s and elements that cant jump to it!). → → $O(N)$!

```
class Solution:
    def canJump(self, nums: List[int]) -> bool:
        goal = len(nums) - 1
        for i in range(goal, -1, -1): #starting from end
            if i + nums[i] >= goal: #if goal can be reached from current i
                goal = i #shifting the goalpost closer!
        return True if goal == 0 else False #goal == starting point at end of a successful run!
```

Memoization: Storing intermediate results so that they don't have to be recalculated again later.

SPACE COMPLEXITY (aka memory usage)

- Non-memoized: $O(n)$
- Memoized: $O(n)$

Majority of the Dynamic Programming problems can be categorized into two types:

2. Combinatorial problems.

Every Dynamic Programming problem has a schema to be followed:

- Show that the problem can be broken down into optimal sub-problems.
- Recursively define the value of the solution by expressing it in terms of optimal solutions for smaller sub-problems.
- Compute the value of the optimal solution in bottom-up fashion.
- Construct an optimal solution from the computed information.

Problems

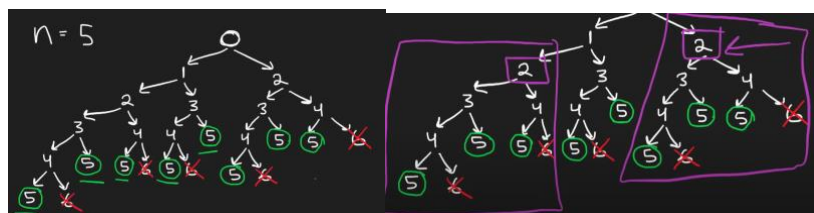
Climbing Stairs

You are given an integer n representing the number of steps to reach the top of a staircase. You can climb with either 1 or 2 steps at a time. Return the number of distinct ways to climb to the top.

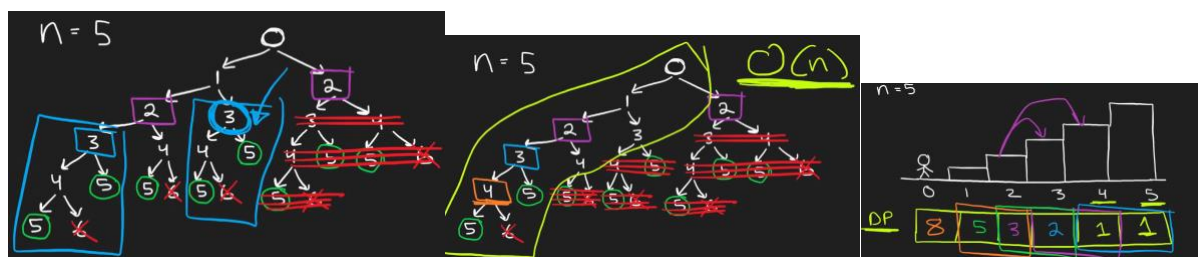
Example 1: Input: $n = 2$; Output: 2; Explanation: 1. $1 + 1 = 2$, 2. $2 = 2$

Example 2: Input: $n = 3$; Output: 3; Explanation: 1. $1 + 1 + 1 = 3$, $1 + 2 = 3$, $2 + 1 = 3$

We can map it out like a decision tree: at every step, the next step can be 1 or 2:



Once we reach 5, we store that path. Else if it goes above n (6), then we return 0. We can see that as you go down the tree, the nodes after you reach 4 are the same; then the same for 3 as well; then the same for 2 too (see purple boxes). This is how we do memoization: once we come to the 2 node, we store its value in a cache, so that next time we encounter that route, we just reuse the value instead of computing it all over again.



If we're doing DFS, we'll go down the left most path first; so the leftmost blue box has been calculated. When we encounter the other 3 node (after the left 1), the value is known, so we just use that instead. Even lower, the 4 node is already calculated in the leftmost branch; it can be reused. Finally, our decision tree to solve just becomes the yellow box – we store all the intermediates! We start at the bottom from the base case (5 at the bottom of the yellow path) and work our way up. – Bottom-up dynamic programming! We make a 'dp' array which at each index stores the number of ways from that step till the end; index corresponds to steps done. We start at the last node itself i.e., n (in this example, 5) and then count the number of ways to reach the end from there (by default 1 for last node). Then from the previous no. of steps i.e., 4 – only 1 way to reach. Then the previous node 3 – 2 ways: $1+1$ or 2; note that it's the sum of the last 2 elements after it – Fibonacci series! See above diagram to understand. Since it's the Fibonacci series (backwards), we don't even need the extra array; we can save the space by using two variables that get added together to get the previous node's value (which gives the number of ways from that node till the end). UNDERSTAND PROPERLY!!

```
class Solution:
    def climbStairs(self, n: int) -> int:
        one, two = 1, 1 #last & second last nodes' number of ways to reach the end
        for i in range(n-1): #steps away from the end
            temp = one
            one = one + two #new value of ith node away from end (immediately to left)
            two = temp #old value of one; shifted leftwards

        return one #Now the first element of dp array; has total number of ways stored in it!
```

House Robber

You are given an integer array `nums` where `nums[i]` represents the amount of money the *i*th house has. The houses are arranged in a straight line, i.e. the *i*th house is the neighbor of the (*i*-1)th and (*i*+1)th house.

You are planning to rob money from the houses, but you cannot rob **two adjacent houses** because the security system will automatically alert the police if two adjacent houses were *both* broken into.

Return the *maximum* amount of money you can rob **without** alerting the police.

Example 1: Input: `nums = [1,1,3,3]`; Output: 4; Explanation: `nums[0] + nums[2] = 1 + 3 = 4`.

Example 2: Input: `nums = [2,9,8,3,6]`; Output: 16; Explanation: `nums[0] + nums[2] + nums[4] = 16`.

We can break this down into recursive subproblems: Say we start from the 1st house, then we need to get the maximum from the sum of 3rd house and onwards. Or, if we skip house 1 and start from the 2nd house, we need to get the maximum sum from the 2nd house onwards (another subarray).

Basically to get the maximum amount to rob, we can write it as: `rob = max(arr[0] + rob(2:n), rob[1:n])` i.e., maximum of starting from first or starting from second.

```
class Solution:
    def rob(self, nums: List[int]) -> int:
        #at every house i, we can then choose i+2 or onwards depending on which path till the end gives the max
        #start from last or second last house - every chain must end in either of these
        rob1, rob2 = 0, 0 #we compare these to get max; they represent the house we robbed before & latest one robbed resp.
        #[rob1, rob2, n, n+1, ...] -> if we want to rob n, we can't use rob2; and we cant use n if we want rob2
        for n in nums:
            temp = max(n + rob1, rob2) #max(current house+rob sum from 2 houses back, current rob sum)
            #if left is greater, then we're using the current node and then moving two steps ahead
            #else if rob2 is greater, we're skipping the current house and moving to the next one!
            rob1 = rob2 #as we're moving to n+1, rob2 is our new rob1!
            rob2 = temp #and rob2 is now updated! Remember, it's the one immediately before current n
        return rob2
```

EXTREMELY IMPORTANT!

- **rob1: Maximum money we can rob from houses up to 2 positions behind current**
- **rob2: Maximum money we can rob from houses up to 1 position behind current**

When we compute `max(n + rob1, rob2)`, we're actually saying: "Rob current house + best possible sum from ≥ 2 houses back" vs "Best possible sum from ≥ 1 house back (which skips current house)"

House Robber 2

Same as above, except with the constraint that the houses are arranged on a circle – so we can't use first and last ones together in the same path since they're adjacent! So it's a simple workaround – we do the above stuff twice: once for `nums[:-1]` (excluding *n*) and for `nums[1:]` (excluding 1), then `max`!

Also, include just the first element as well in the case that there's just one house.

```
class Solution:
    def rob(self, nums: List[int]) -> int:
        return max(nums[0], self.helper(nums[:-1]), self.helper(nums[1:]))

    def helper(self, nums): #does stuff from house robber 1
        rob1, rob2 = 0, 0
        for n in nums:
            temp = max(rob1 + n, rob2)
            rob1 = rob2
            rob2 = temp
        return rob2
```

Longest Palindromic Substring

Given a string *s*, return the longest substring of *s* that is a *palindrome*. A **palindrome** is a string that reads the same forward and backward. If there are multiple palindromic substrings that have the same length, return any one of them.

Example 1: Input: *s* = "ababd"; Output: "bab"; Explanation: Both "aba" and "bab" are valid answers.

Example 2: Input: *s* = "abbc"; Output: "bb"

Got this one by myself, and even more optimal solution than DP! :) $O(n^2)$; Code is self-explanatory:

```
class Solution:
    def longestPalindrome(self, s: str) -> str:
        #get center of a palindrome - check for two consecutive letters (even length), or single letter b/w the same letters (odd length)
        #then expand leftwards & rightwards in both directions and keep checking if the letters are equal; get the count as well

        start = 0 #to get starting position of the largest palindrome - so u can get substring from s using this later
        largest = 1 #for comparing lengths

        def expand_around_center(left, right): #takes left and right indices
            while left >= 0 and right < len(s) and s[left] == s[right]:
                left -= 1
                right += 1 #while loop, ends up forming the entire palindrome!
            return right - left - 1 #returns length of this palindrome formed

        for i in range(len(s)):
            len1 = expand_around_center(i,i) #odd-length: single char is the center
            len2 = expand_around_center(i,i+1) #even-length: 2 consec. chars are the center
            maxlen = max(len1, len2)
            if maxlen > largest:
                largest = maxlen
                start = i - (maxlen - 1) // 2 #center is halfway from start!

        return s[start: start + largest]
```

Palindromic Substrings

Same as above, but now return the total no. of palindromes instead. I used the same code, works

Example 1: Input: *s* = "abc"; Output: 3; Explanation: "a", "b", "c".

Example 2: Input: *s* = "aaa"; Output: 6; Explanation: "a", "a", "a", "aa", "aa", "aaa". Note that different substrings are counted as different palindromes even if the string contents are the same.

```
class Solution:
    def countSubstrings(self, s: str) -> int:
        totalCount = 0

        def expand_around_center(left, right): #takes left and right indices
            count = 0
            while left >= 0 and right < len(s) and s[left] == s[right]:
                left -= 1
                right += 1 #while loop, ends up forming the entire palindrome!
                count += 1
            return count

        for i in range(len(s)):
            count1 = expand_around_center(i,i) #odd-length: single char is the center
            count2 = expand_around_center(i,i+1) #even-length: 2 consec. chars are the center
            totalCount = totalCount + count1 + count2

        return totalCount
```

Decode Ways

A string consisting of uppercase english characters can be encoded to a number using the following mapping: 'A' -> "1", 'B' -> "2" ... 'Z' -> "26"

To **decode** a message, digits must be grouped and then mapped back into letters using the reverse of the mapping above. There may be multiple ways to decode a message. For example, "1012" can be mapped into:

- "JAB" with the grouping (10 1 2)
- "JL" with the grouping (10 12)

The grouping (1 01 2) is invalid because 01 cannot be mapped into a letter since it contains a leading zero. Given a string *s* containing only digits, return the number of ways to **decode** it. You can assume that the answer fits in a **32-bit** integer.

Example 1: Input: *s* = "12"; Output: 2; Explanation: "12" could be decoded as "AB" (1 2) or "L" (12).

Example 2: Input: *s* = "01"; Output: 0; Explanation: "01" cannot be decoded because "01" cannot be mapped into a letter.

For recursion/DP, to come up with a recurrence relation, form a decision tree! At each step, either this or that...

Here at each step, we can take either one or two numbers; we start from the last number digit – the way including that is the starting case number of ways, 1. Then we go towards the start by considering both single and double digits and skipping 0's unless they're after a 1 or 2

```
class Solution:
    def numDecodings(self, s: str) -> int:
        #base case: break down to single characters, single digit: 1 way if 1-9; 0 ways if 0
        #Recurrence relation: at each step we can take the next 1 digit or the next 2 digits if in 10-26
        dp = [len(s) : 1] #no. of ways to decode string after index i => stores these subproblems
        for i in range(len(s)-1, -1, -1): #start from end - note second -1 is so that it goes till index 0!
            if s[i] == '0':
                dp[i] = 0 #invalid if starting with 0; note that 10/20 gets accounted for below
            else:
                dp[i] = dp[i+1] #next single digit; adds one way?
                if (i+1 < len(s) and (s[i] == '1' or (s[i] == '2' and s[i+1] in '0123456'))):
                    #if there's a second character after current, then if current 1 then it can have second digit as anything,
                    #but if current is 2 then second digit should only be in 0-6 range since max is 26!
                    dp[i] += dp[i+2] #including ways with double digit!
        return dp[0] #all ways gets stored at start, like in climbing stairs
```

Each *dp[i]* represents the total number of valid decodings from that position onwards, considering all possible branching paths. The below is a nice example:

Visual Representation:

text

String: "1012"

Index: 0123

dp[4] = 1 (empty string)

dp[3] = 1 (decode "2": one way)

dp[2] = 2 (decode "12": "1,2" OR "12" - two ways)

dp[1] = 0 (can't decode starting with "0")

dp[0] = 2 (decode "1012": must use "10" + decode "12" - two ways)

Coin Change

You are given an integer array `coins` representing coins of different denominations (e.g. 1 dollar, 5 dollars, etc) and an integer `amount` representing a target amount of money. Return the fewest number of coins that you need to make up the *exact* target amount. If it is impossible to make up the amount, return -1. You may assume that you have an unlimited number of each coin.

Example 1: Input: `coins = [1,5,10]`, `amount = 12`; Output: 3

Explanation: $12 = 10 + 1 + 1$. Note that we do not have to use every kind coin available.

Example 2: Input: `coins = [2]`, `amount = 3`; Output: -1

Explanation: The amount of 3 cannot be made up with coins of 2.

Example 3: Input: `coins = [1]`, `amount = 0`; Output: 0

Explanation: Choosing 0 coins is a valid way to make up 0.

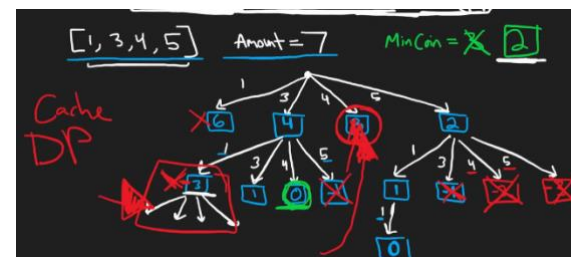
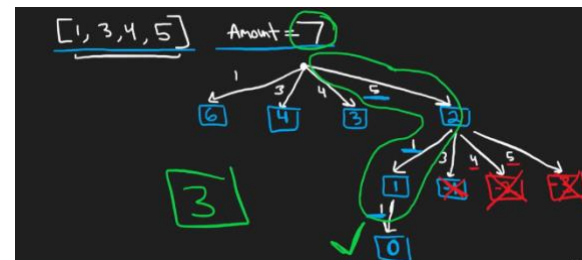
In this decision tree, we can start with any of the given coins that are lesser than the amount; eg: `[1,3,4,5]` → 4 branches! at end of each, subtract them from the amount, then remainder becomes start of another 4-branch node, like above, which shows 3 coins

We can then do all other paths and check which gets to 0 first; that will be the min no. of coins needed. We can use top-down memoization to save all the intermediate results (i.e., when remainder-element = certain value):

In bottom-up DP, we solve this in reverse order; we want to know the minimum number of coins to sum to a particular value:

Eg.: $DP[0] = 0$ (0 coins to sum to 0); $DP[1] = 1$ (just the coin 1),

Then $DP[2] = 1$ (just coin of 1) + $DP[1]$. Why? → see green path in top image; $2 \rightarrow 1 \rightarrow 0$; $DP[1]$ stores no. of coins to get to 0 from 1; and there's one coin to go from 2 to 1. DRAW DECISION TREE!!!



```
class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:

        def dfs(amount):
            if amount == 0:
                return 0

            res = 1e9
            for coin in coins:
                if amount - coin >= 0:
                    res = min(res, 1 + dfs(amount - coin))
            return res

        minCoins = dfs(amount)
        return -1 if minCoins >= 1e9 else minCoins
```

$dp[i]$ = the minimum number of coins required to make up an amount i (see dfs code on left to understand better)

To calculate $dp[i]$, think about the last coin you would add to reach the total i . This last coin must be one of the denominations from our coins array, let's call it c .

If we use a coin c , the remaining amount we need to make is $i - c$. We already know (or can calculate) the minimum number of coins needed for $i - c$, which is $dp[i - c]$. So, the total number of coins in this case would be $dp[i - c] + 1$.

Since we want the *fewest* total coins, we should try this for every possible coin c and choose the option that gives us the minimum result.

This gives us our recurrence relation:

$dp[i] = \min(dp[i - c] + 1)$ for every coin c in `coins` where $i - c \geq 0$.

Base Case:

The simplest case is dp . How many coins do we need to make an amount of 0? Zero coins. So, $dp = 0$.

```

class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        # dp[i] will store the minimum number of coins for amount i.
        # We initialize with a value larger than any possible answer.
        # amount + 1 is a good "infinity" because the max coins can't exceed amount (if coin 1 exists).
        dp = [amount + 1] * (amount + 1)
        dp[0] = 0 # Base case: 0 coins are needed to make an amount of 0.

        for i in range(1, amount + 1): # Iterate through all amounts from 1 to the target amount.
            for coin in coins: # For each amount, try every coin - see how many coins needed to make it
                if i >= coin: # If the coin value is less than or equal to the current amount.
                    # Update dp[i] with the minimum of its current value or using the current coin (1 + the solution for the remaining amount).
                    dp[i] = min(dp[i], 1 + dp[i - coin]) #ensures the least no. of coins is used to get amount i!

        if dp[amount] == amount + 1: # If dp[amount] is still our "infinity" value, it means the amount cannot be made.
            return -1
        else:
            return dp[amount]

```

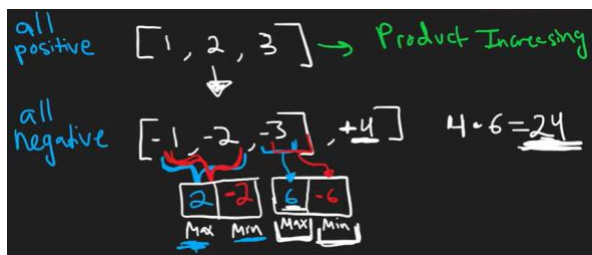
Makes sense after a while – read properly!

Maximum Product Subarray

Given an integer array nums, find a **subarray** that has the largest product within the array and return it. A **subarray** is a contiguous non-empty sequence of elements within an array. You can assume the output will fit into a **32-bit** integer.

Example 1: Input: nums = [1,2,-3,4]; Output: 4 | **Example 2:** Input: nums = [-2,-1]; Output: 2

Similar to maximum subarray greedy problem (kadane)



if all +ve -> keep on multiplying

But with negatives in play, we instead have to store 2 values that get multiplied by the element at each index: the maximum +ve product so far, and the maximum -ve product so far, so that in case the next is negative we can multiply with the negative product and get a large product. Edge case is when we meet 0; break by resetting both max and min to 1 and 1; essentially starting a new substring.

```

class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        res = nums[0] #placeholder
        curMin, curMax = 1, 1 #unitary default multiplier
        for num in nums:
            temp = curMax * num
            curMax = max(num * curMax, num * curMin, num)
            #max of +ve prod, -ve prod, and num itself which could be bigger than either of the products so far
            curMin = min(temp, num * curMin, num)
            #using temp cause we're updating currentMax, but we need the old one from before this num; hence temp
            res = max(res, curMax) #update at each num
        return res

```

Understand properly!